Clasificación automática de 104 tipos de flores con redes neuronales profundas*

Angel Eduardo-Luna[a,1], Andrea Estefanía De la Peña Contreras[a,2], Jannette Cristina Flores Rodríguez[a,3], Rey Iván de la Fuente Chávez[a,4] and Andrés Huerta Robinson[a,5]

[a]*Escuela de Ingeniería y Ciencias, Tecnológico de Monterrey, Monterrey, Nuevo León, México*

INFORMACIÓN

Palabras clave:
Clasificación de imágenes
Redes neuronales convolucionales
Regularización
F1 macro
Flores

Resumen

Se comparan una red totalmente conectada de referencia, una CNN original de cuatro convoluciones, una variante con dos convoluciones adicionales y una intervención de regularización con AdamW y ajuste de tasa. Se utilizan 12,753 imágenes reales de entrenamiento y 3,712 de validación de 104 categorías de flores, con entrada 112×112. Se incluyen representaciones de las arquitecturas, curvas de entrenamiento y validación, matrices de confusión y predicciones ilustrativas. El modelo elegido por F1 macro de validación es CNN + AdamW + LR, con F1 macro 0.5745 y exactitud 64.44 %. La red totalmente conectada es una referencia reconstruida, no un checkpoint histórico identificado del Avance 3. Los resultados corresponden a una sola semilla y no se presentan como evaluación sobre prueba independiente.

1. Resumen y objetivos

Este avance compara una red totalmente conectada de referencia con la CNN original del equipo y con una segunda CNN que agrega dos capas convolucionales. Además, evalúa una intervención de regularización sobre la arquitectura original. El objetivo es medir si extraer patrones espaciales mejora la clasificación de 104 categorías de flores y si aumentar la profundidad o regularizar mejora la generalización. Se reportan exactitud, F1 macro, pérdida y matrices de confusión sobre la misma partición de validación.

Para responder a la observación sobre las representaciones de los modelos, se incorporan diagramas de arquitectura con dimensiones, curvas comparables de entrenamiento y validación, y ejemplos de aciertos y errores. El notebook adjunto contiene las llamadas de entrenamiento y las salidas de la evaluación.

La red totalmente conectada se reconstruye para esta comparación a partir del cargador y del ciclo de entrenamiento existentes. No se encontró en el repositorio un modelo totalmente conectado identificado como la entrega del Avance 3; por ello no se atribuyen a aquella entrega los resultados de esta referencia.

2. Contexto del problema

Muchas categorías de flores son visualmente parecidas y distinguirlas puede requerir conocimiento especializado. Identificarlas de forma manual es lento cuando se tienen miles de fotografías. Por eso resulta útil contar con modelos que reconozcan la clase de una flor a partir de una imagen.

Este proyecto parte de la competencia "Petals to the Metal", encontrado en Kaggle, cuyo reto es construir un modelo que identifique el tipo de flor en un conjunto de imágenes. La competencia se limita a 104 clases de flores, y consiste en un problema de clasificación multiclase de imágenes en el que el modelo debe tener la capacidad de identificar valores finos como la forma, estructura y color para distinguir entre categorías cercanas.

Este problema tiene varias dificultades que lo hacen relevante desde un enfoque de redes neuronales. Primeramente, las clases son heterogéneas, ya que algunas de ellas solo tienen un subtipo de flor y otras agrupan muchas subvariedades. Además, las imágenes de las flores no fueron capturadas en el mismo entorno, dando más variedad a las mismas. Por último, el número de imágenes por cada clase tampoco es uniforme, por lo que, considerando la métrica de evaluación, es un punto fuerte a tomar en cuenta que el modelo funcione de manera efectiva tanto para clases con pocos ejemplos como también para aquellas con muchos.

3. Descripción del dataset

3.1. Origen

Los datos vienen de *Petals to the Metal – Flower Classification on TPU*, una competencia *Getting Started* de Kaggle cuyo objetivo es aprender a entrenar modelos en TPU (Kaggle, s.f.). Según la descripción de la competencia, las imágenes se tomaron de cinco conjuntos de datos públicos. Algunas clases son muy específicas (por ejemplo, *pink primrose*, que contiene un solo subtipo de flor) y otras agrupan muchas subvariedades (por ejemplo, *wild rose*).

Al revisar la lista de clases nos dimos cuenta de que las primeras 102 siguen exactamente el orden del conjunto Oxford 102 Flowers (Nilsback y Zisserman, 2008): 94 conservan el mismo nombre y las otras 8 se renombraron, se corrigieron o se ampliaron (por ejemplo, *oxeye daisy* pasó a ser *daisy* y *bearded iris* pasó a ser *iris*). Las dos clases extra

* Situación problema: Red convolutiva, reporte final. Diseño de Redes Neuronales y Aprendizaje Profundo (TC2035.201). Basado en la competencia *Petals to the Metal – Flower Classification on TPU* (Kaggle).

[1] Matrícula: A00842657
[2] Matrícula: A00840973
[3] Matrícula: A01255711
[4] Matrícula: A01709445
[5] Matrícula: A00838626

Tabla 1
Particiones del conjunto de datos.

Partición	Imágenes	%	¿Etiquetado?
Entrenamiento	12,753	53.5	Sí
Validación	3,712	15.6	Sí
Prueba	7,382	31.0	No
Total	23,847	100.0	

son *common tulip* y *wild rose*. Las etiquetas están en inglés, así que usamos los nombres originales en cursiva.

3.2. Tamaño y estructura

En total hay 23,847 imágenes a color (RGB) de 104 clases, repartidas como muestra la Tabla 1. Vienen en formato TFRecord, que junta muchas imágenes en pocos archivos binarios para que la TPU las lea rápido. Cada registro tiene tres campos: `image` (la foto codificada en JPEG), `id` (un identificador único) y `class` (la etiqueta, un entero de 0 a 103). Los registros de prueba no traen `class`, porque esas etiquetas solo las tiene Kaggle para calificar.

Cada partición viene en 4 resoluciones: 192×192, 224×224, 331×331 y 512×512 píxeles, con 16 archivos TFRecord por partición, por lo que la base de datos trae 192 archivos TFRecord (4 resoluciones × 3 particiones × 16 archivos) más `sample_submission.csv`. Tener varias resoluciones nos conviene porque no todas las arquitecturas se diseñaron para el mismo tamaño de entrada: ResNet50, DenseNet201 y EfficientNetB0 hacen uso de 224 × 224, mientras que las EfficientNet más grandes usan resoluciones mayores. Para dimensionar el problema, una sola imágen de 224 × 224 ya tiene 150,528 valores.

En las ejecuciones de este avance se leen los TFRecord de 224×224, pero se redimensionan las imágenes a 112×112 antes de entrar a cualquiera de los modelos. La resolución de origen y la entrada efectiva de entrenamiento son, por tanto, distintas. Se conservan las particiones y etiquetas originales.

3.3. Ejemplos de las imágenes

La tabla 2 muestra algunas clases concretas y lo que cada una nos dice sobre el problema. Además, dentro de una misma clase, las fotos cambian mucho de escala, pose e iluminación (Nilsback and Zisserman, 2008)

La Figura 1 muestra fotos reales del conjunto de entrenamiento.

3.4. Balance de clases y métrica de evaluación

Con 12,753 imágenes de entrenamiento y 104 clases, tenemos en promedio 123 imágenes por clase (12,753/104 ≈ 122.6). Sin embargo, las clases no tienen el mismo tamaño: como unas clases agrupan muchas subvariedades y otras solo una, algunas tienen muchas más imágenes que otras.

Por este desbalance, la exactitud no basta: un modelo puede acertar mucho en las clases grandes, fallar en las clases pequeñas y aún verse bien. Por esto mismo, Kaggle califica con el F1 macro (Kaggle, s.f.). Para cada clase c se calcula la precisión P_c, la sensibilidad (*recall*) R_c y su media armónica:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{1}$$

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{104}\sum_{c=0}^{103} F1_c, \tag{2}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos, y falsos negativos de la clase c . Así todas las clases pesan lo mismo, sin importar cuántas imágenes tengan.

3.5. Implicaciones para el proyecto

Como Kaggle ya separa entrenamiento y validación, no necesitamos hacer nuestra propia partición: entrenaremos con `train`, elegiremos modelos e hiperparámetros con `val` y usaremos `test` solo para la predicción final. Las imagenes vienen en JPEG con valores de 0 a 255, así que habrá que decodificarlas y escalarlas (a [0, 1] o con el preprocesamiento que pida cada red preentrenada). Como las etiquetas son enteros, la red producirá 104 valores de salida, uno por cada clase, que podrán transformarse en probabilidades mediante Softmax. El entrenamiento se realizará utilizando entropía cruzada categórica dispersa.

4. Algoritmos y aplicaciones

Dado el problema planteado, su solucion implica el uso de algoritmos de vision por computadora.

4.1. Redes Neuronales Convolucionales (CNNs)

Las redes neuronales convolucionales (CNN) son una alternativa especialmente adecuada para este problema debido a su capacidad para procesar información espacial. Mediante filtros convolucionales pueden aprender automáticamente características de las imágenes, como bordes, texturas y formas. En las capas internas pueden emplearse funciones de activación como ReLU, mientras que Softmax puede utilizarse en la salida para representar las probabilidades de las distintas clases.

Este método parece ser el principal y el mejor para producir resultados optimos, aunque, dado a la gran cantidad de datos que se tienen que tratar y la variedad de operaciones que se llevan a cabo, hay que considerar que es un metodo lento y computacionalmente pesado. Esto habilita la posibilidad de usar otros algoritmos que se ajusten a la necesidad del problema.

4.2. Otros algoritmos sin aprendizaje profundo

La Figura 3 de la sección de metodología complementa el esquema conceptual anterior con las capas y dimensiones de los modelos realmente entrenados. ResNet, AlexNet, Inception y los métodos tradicionales que se describen a continuación son alternativas de contexto; no se reportan como modelos ejecutados en esta comparación.

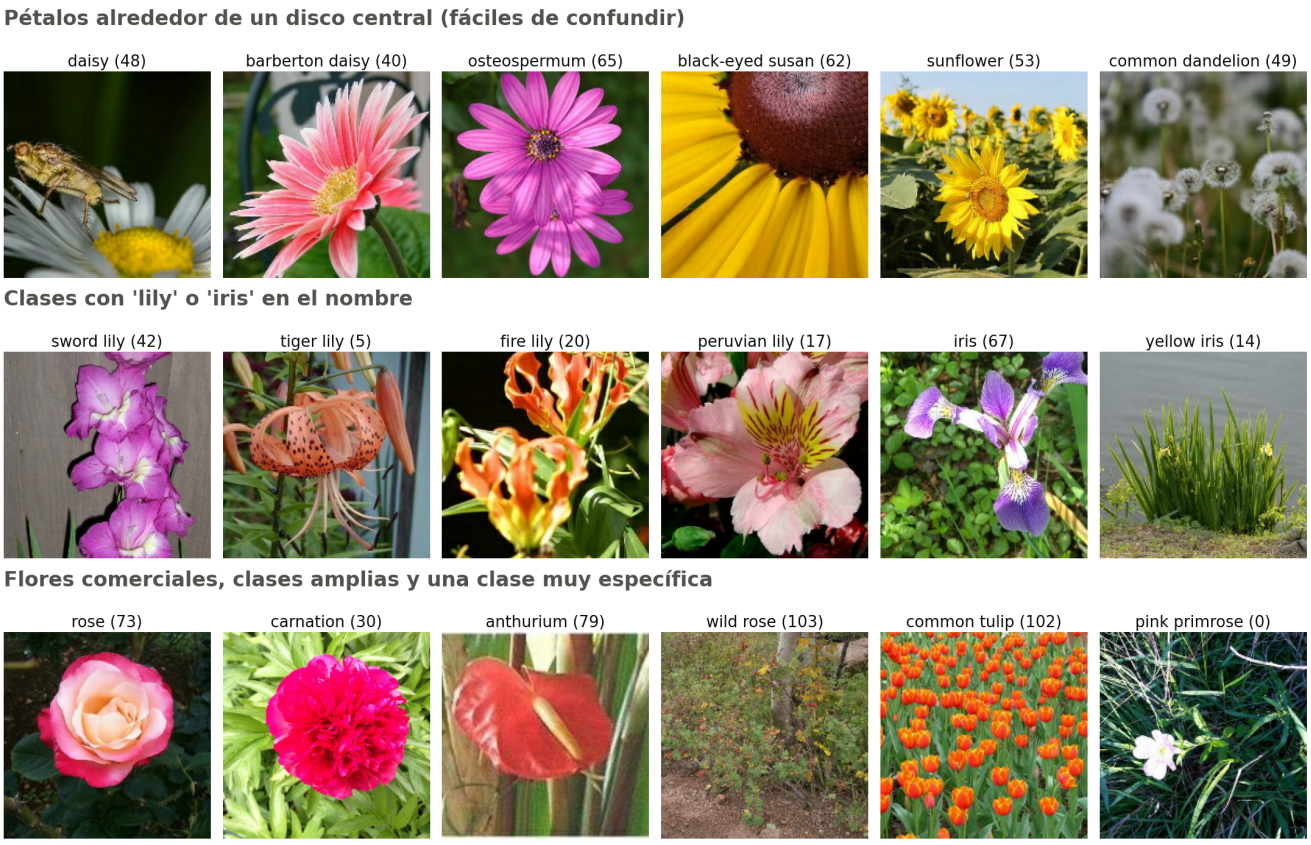


Figura 1: Ejemplos reales del conjunto de entrenamiento con su clase y número de etiqueta.

Tabla 2
Clases de ejemplo (nombre y etiqueta) más lo que ilustran.

Clase (etiqueta)	Que ilustra
sunflower (53)	Girasol. Flor grande y muy característica. Posiblemente sea de las más fáciles de identificar.
daisy (48), *barberton daisy* (40), *osteospermum* (65), *black-eyed susan* (62)	Margarita, gerbera, osteospermum y rudbeckia. Todas tienen pétalos alrededor de un disco central. Pueden surgir confusiones entre ellas.
sword lily (42), *tiger lily* (5), *fire lily* (20), *peruvian lily* (17)	Gladiola, lirio tigre, gloriosa y alstroemeria. Todas dicen *lily*, pero son de cuatro familias botánicas distintas: el modelo tiene que fijarse en la forma, no en el nombre.
iris (67), *yellow iris* (14)	Misma forma general, la diferencia principal es el color.
rose (73), *wild rose* (103)	Dos clases de rosas. *Wild rose* es amplia (varias subvariedades) y no viene de Oxford 102.
common tulip (102)	Tulipán. Clase amplia que tampoco viene de Oxford 102.
pink primrose (0)	Clase muy específica con un solo subtipo de flor.
carnation (30)	Clavel, una de las flores comerciales que se producen en el Estado de México.

k vecinos más cercanos (KNN) Cada imagen lo representa como un vector de características. Usa normalmente la distancia euclidiana para medir las similaridades entre los puntos, y compara con sus k vecinos más cercanos para clasificar la imagen. Es un algoritmo de aprendizaje supervisado.

Bosque aleatorio Se compone de un conjunto de árboles de decisión, se calcula la media de los resultados de los árboles.

Máquinas de Vectores de Soporte (SVMs) Este algoritmo encuentra la línea óptima para separar dos clases. Una variación llamada SVM multiclase existe para problemas multiclase, como este contexto.

4.3. Otros modelos de aprendizaje profundo

ResNet Conocida también como *Residual network*. Útil para entrenar redes neuronales muy profundas. La cota superior para implementar este método es la capacidad de nuestro *hardware* o acceso a equipo

computacional que sea capaz de entrenar este tipo de redes más complejas.

Otros modelos ya implementados Como **AlexNet** o **GoogleNet/inception** que tienen una gran capacidad para reconocer y clasificar imágenes.(Clark, 2025)

4.4. Aplicaciones posibles

Un modelo capaz de clasificar tipos de flores a partir de imágenes puede tener distintas aplicaciones reales. Una de ellas es la **identificación automática de plantas**, donde un usuario podría tomar una fotografía y recibir como resultado la especie o clase más probable.

Un ejemplo de este tipo de aplicaciones yace en el apoyo a plataformas de ciencia ciudadana y el monitoreo de biodiversidad. Por ejemplo, iNaturalist ha acumulado más de 300 millones de observaciones hechas por personas comunes (iNaturalist, 2026) y utiliza un modelo de visión por computadora, entrenado con esas mismas observaciones, para sugerir la especia a partir de una fotografía. Sin este apoyo automático, cada observación dependería de que una persona la revisara detenidamente, lo que haría el proceso lento y difícil de escalar. Automatizar parte de la tarea ayuda a mejorar la eficiencia y aporta de gran manera a la obtención de datos sobre biodiversidad. Este proyecto es una versión a menor escala de este tipo de sistemas.

Otra aplicación sería como **herramienta educativa**, por ejemplo, en aplicaciones para estudiantes o visitantes de jardines botánicos que permitan reconocer flores y consultar información sobre ellas.

Sin embargo, el modelo desarrollado solo podría reconocer las 104 clases presentes en el conjunto de datos, por lo que una aplicación real más amplia requeriría entrenarlo con una mayor variedad de especies.

5. Metodología

5.1. Representación de las arquitecturas implementadas

La Figura 3 representa los tres diseños entrenados desde cero. La entrada efectiva es una imagen RGB de 112×112 píxeles, obtenida de los TFRecord de 224×224 mediante redimensionamiento bilineal con antialias.

La red **totalmente conectada de referencia** aplana los $112 \times 112 \times 3$ valores de entrada en un vector de 37,632 componentes. Aplica una capa densa de 256 unidades con ReLU, dropout de 0.5 y una salida de 104 logits. No contiene convoluciones ni pooling. Utiliza el mismo cargador, aumentos y ciclo de entrenamiento que la CNN.

La **CNN original** emplea cuatro convoluciones de 3×3, padding *same*, ReLU y 32, 64, 128 y 256 filtros. Cada convolución va seguida de max pooling de 2×2. Las salidas de esos bloques son $56 \times 56 \times 32$, $28 \times 28 \times 64$, $14 \times 14 \times 128$ y $7 \times 7 \times 256$. Después aplica global average pooling (GAP), una capa densa de 256 unidades con ReLU, dropout de 0.5 y 104 logits de salida.

La **CNN ampliada** agrega dos capas convolucionales ocultas: una de 128 filtros después del tercer pooling y otra de 256 después del cuarto. Ambas usan kernel 3×3, padding *same* y ReLU, sin pooling adicional. Por tanto, esta versión tiene seis convoluciones. No se trata de dos capas densas nuevas.

La convolución comparte los pesos del filtro entre posiciones y analiza vecindarios de píxeles. Así puede aprender bordes, texturas y partes de pétalos antes de combinarlos. Una capa densa sobre píxeles aplanados asigna pesos independientes a cada posición y no impone esta estructura local. Esta diferencia motiva la comparación, pero la mejora debe comprobarse con las métricas.

5.2. Preprocesamiento y aumento de datos

Se decodifica cada JPEG como RGB, se convierten sus valores a float32, se normalizan mediante $x_{\text{norm}} = x/255$ y se redimensionan a 112×112. Las etiquetas son enteros entre 0 y 103. Se mantiene la separación de archivos original entre entrenamiento y validación, sin incorporar imágenes de validación al entrenamiento.

Solo en entrenamiento se aplican volteo horizontal, rotación aleatoria de factor 0.05, zoom de 0.1 y contraste de 0.1. Los valores resultantes se recortan a $[0, 1]$. Los aumentos se calculan después de la caché y del mezclado, por lo que cambian entre épocas. La validación usa imágenes deterministas sin aumentos.

5.3. Pérdida y actualización de parámetros

Los modelos producen 104 logits z_k. La entropía cruzada se calcula directamente sobre ellos, sin añadir una segunda softmax a la pérdida:

$$L = -\log\left(\frac{\exp(z_c)}{\sum_{k=1}^{104}\exp(z_k)}\right), \tag{3}$$

donde c es la clase verdadera. Para visualizar la confianza se transforma la salida mediante softmax; esta confianza no constituye una evaluación de calibración.

Todas las redes se optimizan con AdamW (Loshchilov and Hutter, 2019) (decaimiento de pesos desacoplado de 0.01, sin aplicarlo a los sesgos) y una tasa inicial de 10^{-3} que se reduce a la mitad cuando la pérdida de validación no mejora en dos épocas, hasta 10^{-5}. La MLP es la excepción en la tasa: usa 10^{-4} (mínimo 10^{-6}), porque en el Avance 4 un primer intento con 10^{-3} colapsó a predecir una sola clase. Todos los modelos usan lotes de 64 imágenes y se entrenan con TensorFlow 2.18 sobre la GPU de una Mac M4 (`tensorflow-metal`).

5.4. Pesos por clase

Para contrarrestar el desbalance (Sección 7.1), una variante pondera la pérdida de cada imagen según su clase (He and Garcia, 2009):

$$w_c = \frac{N}{K\, n_c}, \tag{4}$$

donde $N = 12{,}753$ es el total de imágenes, $K = 104$ el número de clases y n_c las imágenes de la clase c. Una clase

de tamaño promedio recibe peso 1; las clases de 18 imágenes reciben 6.8 y *iris* (782) recibe 0.16. Como $\sum_c w_c n_c = N$, la escala total de la pérdida no cambia.

6. Protocolo experimental

6.1. Condiciones comunes

Todos los experimentos usan los mismos 12,753 ejemplos de entrenamiento y 3,712 de validación de las 104 clases, la semilla 0 y lotes de 64 imágenes. La *referencia* es la CNN de cuatro convoluciones elegida en el Avance 4: dropout de 0.5 antes de la salida, aumento de datos solo en entrenamiento (volteo horizontal, rotación, zoom y contraste), AdamW con decaimiento de pesos de 0.01 (Loshchilov and Hutter, 2019), tasa inicial de 10^{-3} reducida a la mitad cuando la pérdida de validación no mejora en dos épocas (mínimo 10^{-5}), entrada de 112×112 y hasta 150 épocas. El entrenamiento se detiene tras 12 evaluaciones sin mejorar el F1 macro de validación y se conserva el checkpoint con mayor F1 macro; todas las métricas reportadas corresponden a ese checkpoint, no a la última época.

6.2. Un cambio a la vez

Para atribuir cada diferencia a una causa, cada experimento modifica un solo factor respecto a la referencia: la arquitectura (MLP o CNN ampliada), la pérdida (pesos por clase), la regularización o un hiperparámetro (tasa de aprendizaje o dropout). Hay dos excepciones declaradas. La MLP usa una tasa inicial de 10^{-4}, porque con 10^{-3} colapsó a predecir una sola clase en el Avance 4. La variante *sin regularización* retira a la vez dropout, aumento y decaimiento de pesos (Adam en lugar de AdamW), para medir su efecto conjunto.

6.3. Métricas de entrenamiento comparables con validación

Las métricas que reporta el ciclo de entrenamiento se calculan con aumento y dropout activos, por lo que no son comparables con validación. Para medir la brecha de generalización, en cada época se evalúan además 3,712 imágenes fijas de entrenamiento (el mismo tamaño que validación) sin aumento y con la red en modo inferencia. Con ellas se calculan la pérdida, la exactitud y el F1 macro de entrenamiento. La *brecha F1* es el F1 macro de ese subconjunto menos el de validación, ambos en la época del checkpoint.

6.4. Alcance de la evaluación

La validación se usa para elegir el checkpoint y comparar configuraciones; sus métricas no son una estimación independiente de generalización. Como el conjunto de prueba de Kaggle no tiene etiquetas locales, la medida de prueba se obtiene enviando las predicciones a la competencia (Sección 7.9). Cada configuración se entrenó con una sola semilla, por lo que diferencias de pocas milésimas de F1 macro no se interpretan como mejoras. Los resultados de este documento se generaron con TensorFlow 2.18 en una GPU Apple M4 (`tensorflow-metal`); las corridas del Avance 4 se hicieron con PyTorch/DirectML, por lo que sus cifras no se mezclan con las de este reporte.

6.5. Reproducibilidad

El código, las configuraciones y los resultados están en el repositorio público https://github.com/Lunaaaalj/NeuralNetworksTeamEnvironment. El `README` explica la instalación y los comandos. El notebook `notebooks/reporte_final.ipynb` reconstruye de principio a fin la tabla de resultados (`reports/final/resultados.csv`), las tablas y las figuras de este documento, a partir de los `history.csv`, `summary.json` y `eval_report.json` versionados de cada experimento; con `RUN_TRAINING = True` también vuelve a entrenarlos. La matriz de experimentos está en `scripts/run_final.py` y la configuración de referencia en `configs/final.yaml`.

7. Resultados

Todas las cifras de esta sección provienen de `reports/final/resultados.csv`, que el notebook de la Sección 6.5 reconstruye a partir de los `history.csv` y `eval_report.json` de cada corrida. Salvo que se indique otra cosa, las métricas son de validación (3,712 imágenes) en el checkpoint con mayor F1 macro.

7.1. Desbalance de clases

El conjunto de entrenamiento tiene 12,753 imágenes repartidas de forma muy desigual entre las 104 clases (Figura 4). La clase más pequeña tiene 18 imágenes (*moon orchid*, *bolero deep blue* y *alpine sea holly*) y la más grande 782 (*iris*): una razón de 43:1. La mediana es de 88 imágenes y la media de 122.6; la media queda por encima de la mediana porque unas pocas clases concentran los datos: las 10 más grandes reúnen el 40 % de las imágenes, mientras que 24 clases tienen menos de 30. El rango intercuartílico va de 32.5 a 125.5 imágenes. Validación conserva exactamente las mismas proporciones (de 5 a 228 imágenes por clase), de modo que las clases raras tienen también muy pocos ejemplos para evaluarlas: el F1 de una clase con 5 imágenes de validación cambia en más de 0.15 con un solo acierto.

7.2. Arquitecturas: MLP, CNN de referencia y CNN ampliada

La Tabla 3 resume las arquitecturas y la Figura 5 muestra sus curvas de pérdida, exactitud y F1 macro en entrenamiento y validación. La CNN de referencia alcanza un F1 macro de 0.5784 y una exactitud de 64.06 %; la MLP, con 20 veces más parámetros, se queda en 0.0178 y 14.36 %. La CNN ampliada llega a 0.5663, por debajo de la referencia. Su corrida se interrumpió en la época 50 por un bloqueo de la GPU; como su F1 de validación llevaba ocho épocas estable alrededor de 0.564 y la tasa de aprendizaje ya había bajado a 3×10^{-5}, es poco probable que hubiera alcanzado a la referencia.

Las curvas muestran tres comportamientos distintos:

- **CNN de referencia: sobreajuste moderado y controlado.** Las dos curvas avanzan juntas durante unas

Tabla 3
Arquitecturas comparadas con los mismos datos de validación (3,712 imágenes).

Modelo	Parámetros	Época	Exact. (%)	F1 entr.	Brecha F1	Tiempo (min)	F1 macro
MLP	9,660,776	62/74	14.36	0.0192	+0.0014	6.9	0.0178
CNN de referencia	480,936	91/103	64.06	0.8138	+0.2354	19.7	**0.5784**
CNN ampliada (6 conv.)†	1,218,600	41/49	63.77	0.7936	+0.2273	17.1	0.5663

Época: checkpoint elegido por F1 macro / épocas ejecutadas. F1 entr.: F1 macro sobre 3,712 imágenes fijas de entrenamiento sin aumento ni dropout, en la misma época. Brecha F1 = F1 entr. − F1 macro de validación. Todas a 112×112, entrenadas desde cero con semilla 0. †La corrida se interrumpió en la época 50 por un bloqueo de la GPU; se reporta su mejor checkpoint.

15 épocas y después se separan. La pérdida de validación toca su mínimo en la época 54 (1.464) y después apenas sube (1.486 al final), mientras el F1 de validación sigue mejorando poco a poco hasta la época 91. En el checkpoint, el F1 de entrenamiento es 0.8138 y el de validación 0.5784: una brecha de 0.235.

- **CNN ampliada: el mismo techo, alcanzado antes.** Con 2.5 veces más parámetros aprende el entrenamiento más rápido (en la época 41 su F1 de entrenamiento es 0.794 frente a 0.638 de la referencia), pero su validación se estanca en el mismo nivel. Su pérdida de validación toca su mínimo antes (época 34) y después sube de 1.555 a 1.602, una señal de sobreajuste más clara que en la referencia.

- **MLP: infraajuste.** Entrenamiento y validación casi coinciden (brecha de F1 de 0.001) y ambas son muy bajas. La red no memoriza: no logra aprender. Solo predice 11 de las 104 clases y tiene F1 = 0 en 93.

En las tres redes, la curva gris queda por debajo de la azul: el aumento de datos y el dropout hacen que las métricas del ciclo de entrenamiento parezcan peores de lo que el modelo realmente rinde sobre imágenes de entrenamiento.

La Tabla 4 justifica conservar el checkpoint con mayor F1 macro y no la última época. Entre la época 91 y la 103, el modelo sigue ajustando un poco el entrenamiento (la pérdida baja de 0.557 a 0.551), pero el F1 de validación cae de 0.5784 a 0.5731 y su pérdida sube. La diferencia entre exactitud (64.06 %) y F1 macro (0.578) también es informativa: la exactitud cuenta aciertos sobre todas las imágenes y la dominan las clases grandes; el F1 macro promedia por clase, así que pesa igual una clase de 18 imágenes que *iris* con 782.

7.3. Comparación con el Avance 4

La Tabla 5 contrasta estas corridas con las del Avance 4. El F1 de la referencia se reproduce (0.5745 frente a 0.5784) aunque cambió el *framework*, lo que da confianza en la comparación. Lo que cambia es la brecha: medida con aumento y dropout activos, como en el Avance 4, la brecha de exactitud de la referencia era de 0.105; medida sobre imágenes limpias es de 0.189 en exactitud y de 0.235 en F1 macro. El Avance 4 subestimaba el sobreajuste a casi la mitad. En la CNN ampliada, que en el Avance 4 se entrenó con Adam y tasa fija, la misma política de la referencia

Tabla 4
CNN de referencia en la época elegida y al terminar el entrenamiento.

Métrica	Época elegida (91)	Última época (1
Exactitud de entrenamiento (%)	83.00	8
F1 macro de entrenamiento	0.8138	0.8
Pérdida de entrenamiento	0.557	0.
Exactitud de validación (%)	64.06	6
F1 macro de validación	0.5784	0.5
Pérdida de validación	1.483	1.

Métricas de entrenamiento sobre 3,712 imágenes fijas sin aumento, en modo infer

reduce su desventaja de 0.024 a 0.012 puntos de F1, pero no la elimina.

7.4. F1 por clase de la CNN de referencia

El F1 macro de 0.578 esconde mucha variación entre clases (Tabla 6). La mediana por clase es 0.600; 16 clases superan 0.8 y 13 quedan por debajo de 0.3. Las mejores son flores con forma o color muy característicos (*pink-yellow dahlia* y *cautleya spicata* con F1 de 1.0, *cape flower* y *purple coneflower* con 0.97). Cinco clases tienen F1 = 0: *bolero deep blue*, *siam tulip*, *canterbury bells*, *pink quill* y *garden phlox*. Todas tienen entre 18 y 24 imágenes de entrenamiento y entre 5 y 7 de validación, y el modelo nunca las predice.

El tamaño de la clase explica solo una parte de estos resultados (Figura 7). Las 10 clases más pequeñas tienen un F1 promedio de 0.361 y las 10 más grandes de 0.703, pero la relación no es monótona: las clases de 30 a 60 imágenes promedian 0.732, más que las de 300 o más (0.690), y la correlación de Spearman entre tamaño y F1 es apenas 0.12. Los fallos se concentran en la cola de clases con menos de 30 imágenes (F1 medio 0.461) y en flores parecidas entre sí. Los pares más confundidos son *carnation* → *rose* (13 imágenes), *wild rose* → *windflower* (12), *common dandelion* → *sunflower* (12) y *common tulip* → *rose* (11): flores de forma y color similares, donde la clase predicha suele ser además la más frecuente. La Figura 8 muestra la matriz de confusión completa.

La Figura 6 muestra ejemplos reales. Los errores ilustran en qué se apoya la red: un girasol de pétalos rojos se predice como *tiger lily* y un iris barbado morado como *wild geranium*, es decir, cuando el color se aparta del típico de la clase, la red se guía por el color antes que por la forma. Varios errores tienen además confianza baja (17–42 %), mientras

Tabla 5
Avance 4 frente al reporte final: la brecha medida con aumento subestima el sobreajuste.

Modelo	Política en Avance 4	F1 A4	Brecha exact. A4	F1 final	Brecha exact.	Brecha F1
CNN de referencia	AdamW + LR adaptable	0.5745	+0.1051	0.5784	+0.1894	+0.2354
CNN ampliada (6 conv.)	Adam, LR fijo	0.5502	+0.1687	0.5663	+0.1926	+0.2273
MLP	Adam, LR fijo	0.0280	-0.0346	0.0178	+0.0140	+0.0014

A4: PyTorch/DirectML; su exactitud de entrenamiento se midió durante el ciclo, con aumento y dropout activos. Final: TensorFlow/Metal, todas con AdamW + LR adaptable; brechas medidas sobre 3,712 imágenes de entrenamiento sin aumento y en modo inferencia, en la época del checkpoint.

Tabla 6
F1 por clase de la CNN de referencia: las 10 mejores y las 10 peores.

Mejores	n_{entr}	F1	Peores	n_{entr}	F1
pink-yellow dahlia	58	1.000	bolero deep blue	18	0.000
cautleya spicata	27	1.000	siam tulip	19	0.000
cape flower	57	0.971	canterbury bells	20	0.000
purple coneflower	55	0.968	pink quill	24	0.000
bishop of llandaff	58	0.941	garden phlox	24	0.000
silverbush	28	0.941	tree mallow	29	0.154
californian poppy	55	0.938	sword lily	63	0.200
geranium	63	0.919	hibiscus	134	0.243
orange dahlia	36	0.900	spring crocus	21	0.250
osteospermum	33	0.900	globe-flower	21	0.250

n_{entr}: imágenes de esa clase en entrenamiento. Desempates por tamaño de clase.

Tabla 7
Pérdida ponderada por clase frente a la pérdida sin ponderar.

Entrenamiento	Exact. (%)	F1 10 menores	F1 10 mayores	Clases F1 = 0	F1 macro
CNN de referencia	64.06	0.3607	0.7734	9	0.5784

F1 10 menores/mayores: F1 promedio de las 10 clases con menos (18–19) y más (460–782) imágenes de entrenamiento. Pesos $w_c = N/(K\,n_c)$: van de 0.16 (iris) a 6.81 (las clases de 18 imágenes).

que los aciertos en clases características (*sunflower*, *iris*) llegan a casi 100 %.

7.5. Pesos por clase

7.6. Efecto de la regularización

Para medir qué aportan las técnicas de regularización, se entrenó la misma CNN, con la misma semilla y el mismo tope de épocas, sin dropout, sin aumento de datos y con Adam en lugar de AdamW (Tabla 8 y Figura 9). Sin regularización, la red aprende más rápido al principio: llega a un F1 de validación de 0.5 en la época 25, frente a la 38 de la referencia. Pero ahí se estanca. Su mejor F1 es 0.5332, 0.045 por debajo de la referencia (0.5784), y su exactitud baja de 64.06 % a 59.64 %.

La causa es el sobreajuste. Sin regularización, el F1 de entrenamiento sube a 0.877 (0.814 con regularización) y la brecha de F1 pasa de 0.235 a 0.344. La pérdida de validación toca su mínimo en la época 25 (1.825) y después sube hasta 2.122, mientras que con regularización su mínimo es 1.464 y prácticamente no sube. El daño se concentra en las clases pequeñas: el F1 promedio de las 10 clases más pequeñas baja de 0.361 a 0.257, mientras que el de las 10 más grandes solo baja de 0.703 a 0.669. Las clases con 18–30 imágenes son las más fáciles de memorizar y, sin aumento, la red ve siempre las mismas pocas fotos.

7.7. Hiperparámetros

La Tabla 9 cambia un hiperparámetro a la vez respecto a la referencia, con el conjunto completo de entrenamiento. Ninguna de las dos variantes la mejora:

- **Tasa de aprendizaje de 3×10^{-4} (en vez de 10^{-3}): F1 0.4605.** Es la peor de las CNN, pero no por sobreajuste: su brecha de F1 es solo 0.094 porque la red tampoco aprende bien el entrenamiento (F1 de entrenamiento 0.554, contra 0.814 de la referencia). La reducción automática de la tasa ocurre cuando la pérdida de validación deja de mejorar dos épocas; al partir de una tasa ya baja, en pocas reducciones llega al mínimo de 10^{-5} y el aprendizaje prácticamente se detiene. En la época del checkpoint (98) la tasa ya estaba en ese mínimo. Es infraajuste causado por el calendario de la tasa, no por la capacidad de la red.
- **Dropout de 0.3 (en vez de 0.5): F1 0.5596.** Con menos dropout, el F1 de entrenamiento es prácticamente igual (0.816 contra 0.814), pero el de validación baja 0.019 y la brecha sube de 0.235 a 0.256. Es el mismo patrón que sin regularización, en menor grado: menos regularización, más memorización. De nuevo, las clases pequeñas son las que más pierden (0.286 contra 0.361).

Tabla 8
La misma CNN con y sin regularización (semilla 0, mismas épocas máximas).

Variante	Dropout	Aumento	Pesos	Época	Exact. (%)	F1 entr.	Brecha F1	F1 macro
CNN de referencia	0.5	sí	WD 0.01	91/103	64.06	0.8138	+0.2354	**0.5784**
Sin regularización	0	no	–	34/46	59.64	0.8767	+0.3435	0.5332

Pesos: WD = weight decay desacoplado de AdamW; – = Adam sin decaimiento. Ambas comparten arquitectura, tasa de aprendizaje inicial, reducción de LR en meseta, paciencia de 12 evaluaciones y tope de 150 épocas.

Tabla 9
Efecto de un hiperparámetro a la vez, con el conjunto completo.

Configuración	LR	Dropout	Resolución	Época	Exact. (%)	Brecha F1	Tiempo (min)	F1 macro
CNN de referencia	0.001	0.5	112	91/103	64.06	+0.2354	19.7	**0.5784**
LR 3×10^{-4}	0.0003	0.5	112	98/110	54.18	+0.0936	22.5	0.4605
Dropout 0.3	0.001	0.3	112	61/73	61.96	+0.2561	15.0	0.5596

Cada fila cambia un solo valor respecto a la referencia (LR 10^{-3}, dropout 0.5, 112×112).

Tabla 10
Selección aleatoria frente a destilación con Distribution Matching (DM).

IPC	Imágenes	Conjunto	Exact. (%)	F1 macro	Destilación (min)
1	104	Sintéticas (DM)	9.85	0.0672 ± 0.0052	16
1	104	Reales aleatorias	5.75	0.0514 ± 0.0029	–
10	1,040	Sintéticas (DM)	20.55	0.2044 ± 0.0235	25
10	1,040	Reales aleatorias	25.86	0.2575 ± 0.0082	–

Promedio ± desviación estándar de tres semillas de entrenamiento por condición. Misma CNN, Adam 10^{-3}, 112×112. Datos: `reports/resultados.csv`.

Con una sola semilla, la diferencia de 0.019 del dropout es moderada; la de la tasa de aprendizaje (0.118) es clara.

7.8. Reducción de datos con Distribution Matching

Como experimento independiente de los anteriores, se evaluó si un conjunto pequeño de imágenes sintéticas generado con Distribution Matching (DM) conserva más información que el mismo número de imágenes reales elegidas al azar (Tabla 10). Con una imagen por clase (IPC 1, 104 imágenes), DM supera a la selección aleatoria (F1 0.067 frente a 0.051). Con diez por clase (IPC 10, 1,040 imágenes) ocurre lo contrario: las imágenes reales dan 0.258 y las sintéticas 0.204, con más variación entre semillas. La destilación ayuda solo cuando hay muy pocas imágenes, y su costo (16 y 25 minutos de generación) se suma al del entrenamiento. Ninguna de estas imágenes se usó en los demás experimentos; todos entrenan con las 12,753 imágenes reales.

7.9. Evaluación en Kaggle

8. Análisis

8.1. Por qué la CNN supera a la MLP

La diferencia entre la MLP (F1 0.018) y la CNN de referencia (F1 0.578) no es de grado sino de tipo: la MLP no logra ajustar ni siquiera el entrenamiento (F1 de entrenamiento 0.019), mientras que la CNN sí lo ajusta y luego generaliza parcialmente. La causa está en cómo cada red trata la imagen.

- **La MLP no comparte pesos.** Aplana la imagen en 37,632 valores y conecta cada uno con cada neurona oculta: el 99.7 % de sus 9.66 millones de parámetros está en esa primera capa. Un pétalo en la esquina superior y el mismo pétalo en el centro activan pesos distintos, así que la red tendría que aprender cada patrón por separado en cada posición. Con 122 imágenes por clase en promedio, no hay datos suficientes para eso.
- **La CNN codifica supuestos correctos sobre imágenes.** Sus filtros de 3×3 miran vecindarios locales y se aplican en toda la imagen (pesos compartidos), las capas sucesivas combinan bordes en texturas y texturas en partes, y el *pooling* global hace que la predicción dependa de qué hay en la imagen y no de dónde está. Con 20 veces menos parámetros (480,936) aprovecha mucho mejor los mismos datos.
- **La MLP se refugia en las clases grandes.** Solo predice 11 clases y su F1 por clase sí depende fuertemente del tamaño de la clase (Spearman 0.51, frente a 0.12 en la CNN): sin poder distinguir flores, lo mejor que encuentra es favorecer las clases más frecuentes. Además es inestable: con la tasa de 10^{-3} de la CNN colapsa a una sola clase, por lo que necesitó 10^{-4}.

La comparación no aísla por completo el efecto de las convoluciones: las dos redes difieren también en número de parámetros y la MLP usa una tasa de aprendizaje diez veces menor. Aun así, una diferencia de 0.56 puntos de F1 macro con la misma política de entrenamiento, los mismos datos y el mismo aumento no se explica por la tasa: con la tasa de la CNN, la MLP es todavía peor.

Los regularizadores no son el problema de la MLP: su curva con aumento y dropout (gris en la Figura 5) es todavía peor que la limpia, y su brecha es nula. Regularizar más una red que infraajusta no la ayudaría.

8.2. Por qué la CNN más profunda empeoró

Agregar dos convoluciones (de 480,936 a 1,218,600 parámetros) bajó el F1 de 0.578 a 0.566. Con una sola semilla, una diferencia de 0.012 es pequeña, pero coincide con el Avance 4 (0.574 contra 0.550) y las curvas explican por qué la red más grande no mejora:

- **Más capacidad sin más datos.** La red ampliada ajusta el entrenamiento mucho más rápido: en la época 41 su F1 de entrenamiento es 0.794, contra 0.638 de la referencia, pero su F1 de validación se estanca en 0.564. La capacidad extra se usa para memorizar detalles del entrenamiento, sobre todo en las clases con 18–30 imágenes, no para aprender rasgos que generalicen.
- **El sobreajuste aparece antes.** Su pérdida de validación toca su mínimo en la época 34 (frente a la 54 de la referencia) y después sube, y tiene seis clases con $F1 = 0$ en vez de cinco.
- **La regularización estaba calibrada para la red pequeña.** Ambas usan dropout 0.5 y decaimiento de pesos de 0.01. Para una red con 2.5 veces más parámetros, esa misma regularización es relativamente más débil.
- **Más profundidad no agrega información a** 112×112**.** Las dos capas nuevas operan sobre mapas ya reducidos; a esa resolución, los detalles finos que distinguen flores parecidas (*carnation* contra *rose*) ya se perdieron en la entrada, y ninguna capa extra los recupera.

En resumen, el cuello de botella de este problema no es la capacidad del modelo sino la cantidad de datos por clase. Por eso los cambios que atacan la falta de datos, como el aumento, tienen más efecto que hacer la red más grande.

8.3. Qué aporta la regularización

Los tres experimentos de regularización apuntan en la misma dirección: quitar regularización (Sección 7.6) o reducirla (dropout 0.3) sube el F1 de entrenamiento o lo deja igual, pero baja el de validación y abre la brecha. Con 12,753 imágenes y 480,936 parámetros, la CNN tiene capacidad de sobra para memorizar el entrenamiento; la regularización es la que le permite generalizar. Su efecto conjunto vale 0.045 puntos de F1 macro.

La regularización no elimina el sobreajuste: la referencia conserva una brecha de 0.235. Las curvas muestran que la brecha se estabiliza alrededor de la época 60, justo cuando la reducción automática de la tasa lleva el aprendizaje casi a cero. A partir de ahí la red deja de cambiar, en vez de seguir memorizando. Esto también explica por qué la referencia toma su mejor checkpoint tan tarde (época 91): son ajustes finos con una tasa de 10^{-5}.

8.4. Sensibilidad a los hiperparámetros

La tasa de aprendizaje es, de lejos, el hiperparámetro más sensible: bajarla a 3×10^{-4} cuesta 0.118 de F1, más que quitar toda la regularización. La causa es la interacción con la reducción en meseta: una tasa inicial baja hace que el calendario la lleve al mínimo antes de que la red aprenda lo suficiente. Con este calendario, 10^{-3} es la elección correcta. El dropout es menos sensible: entre 0.3 y 0.5 la diferencia es de 0.019, a favor de 0.5.

8.5. Limitaciones

Cada configuración se entrenó con una sola semilla, por lo que diferencias de menos de unas centésimas de F1 no deben tomarse como concluyentes. La validación se usó para elegir el checkpoint de cada corrida, lo que sesga sus métricas ligeramente hacia arriba; los puntajes de Kaggle (Sección 7.9) son la única medida sobre datos nunca vistos. La corrida de la CNN ampliada se interrumpió en la época 50. Todas las redes se entrenaron desde cero a 112×112; no se evaluó transferencia de aprendizaje desde modelos preentrenados.

9. Conclusiones

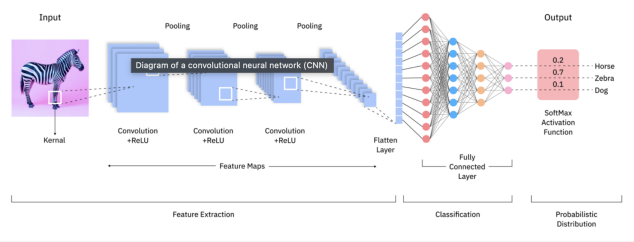


Figura 2: La convolución descompone la imagen en caracteristicas para aprender a clasificar (Clark, 2025).

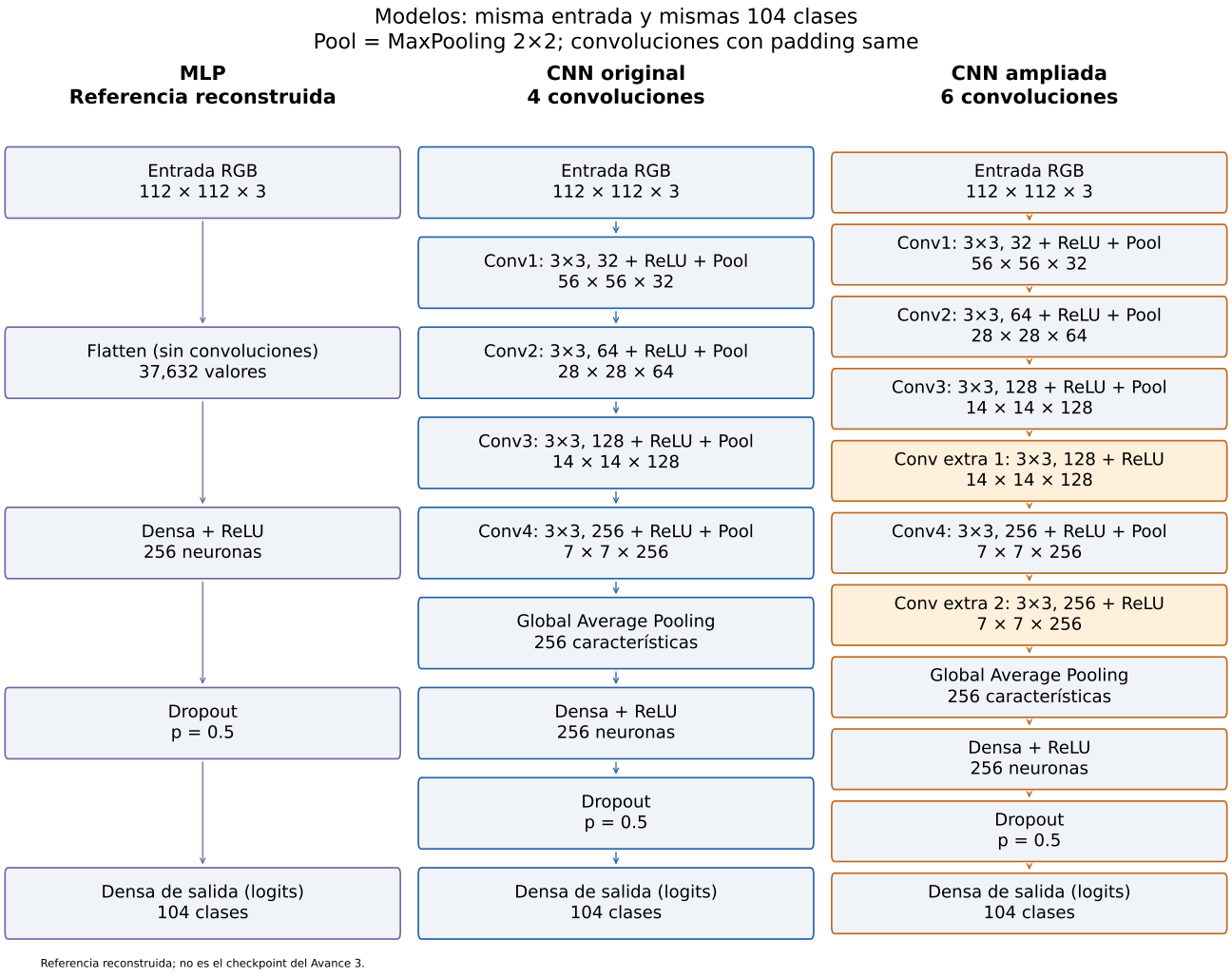


Figura 3: Representación de la red totalmente conectada de referencia, la CNN original y la CNN con dos convoluciones adicionales. Las dimensiones corresponden a las salidas de cada etapa; las capas adicionales se destacan en naranja. La variante regularizada conserva la arquitectura de la CNN original.

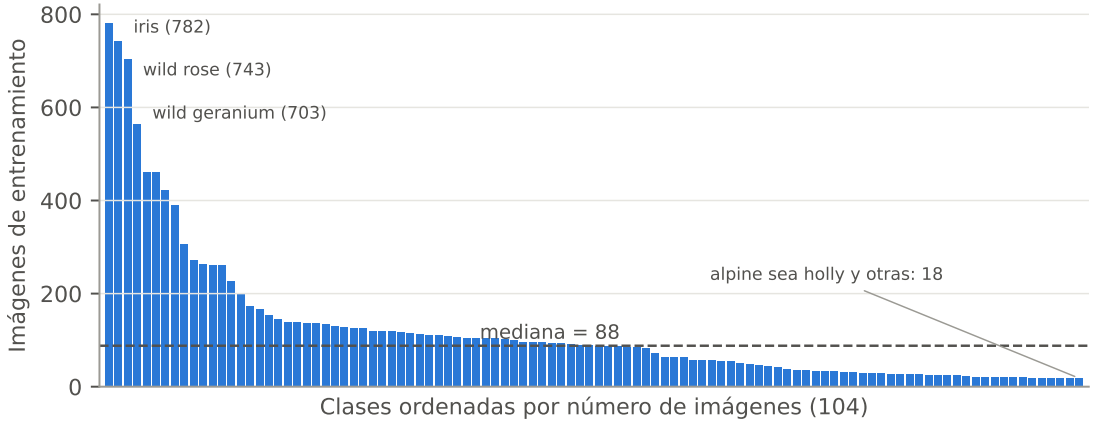


Figura 4: Imágenes de entrenamiento por clase, ordenadas de mayor a menor. La línea punteada marca la mediana (88).

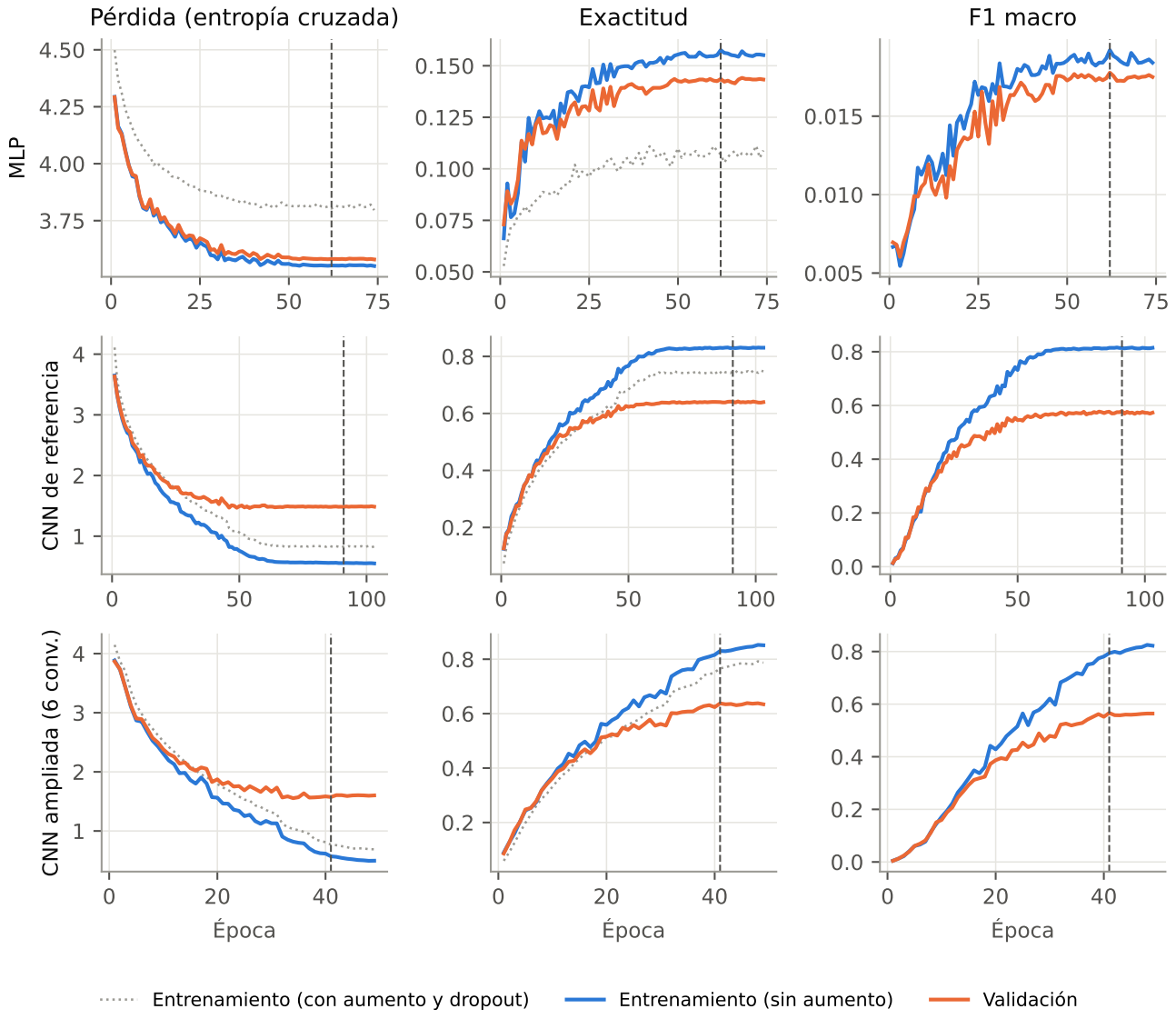


Figura 5: Curvas de entrenamiento y validación por época. Azul: 3,712 imágenes fijas de entrenamiento sin aumento, en modo inferencia. Naranja: validación. Gris punteado: la métrica que reporta el ciclo de entrenamiento, con aumento y dropout activos (no existe para el F1). La línea vertical marca el checkpoint elegido. Las escalas verticales de la MLP son distintas.

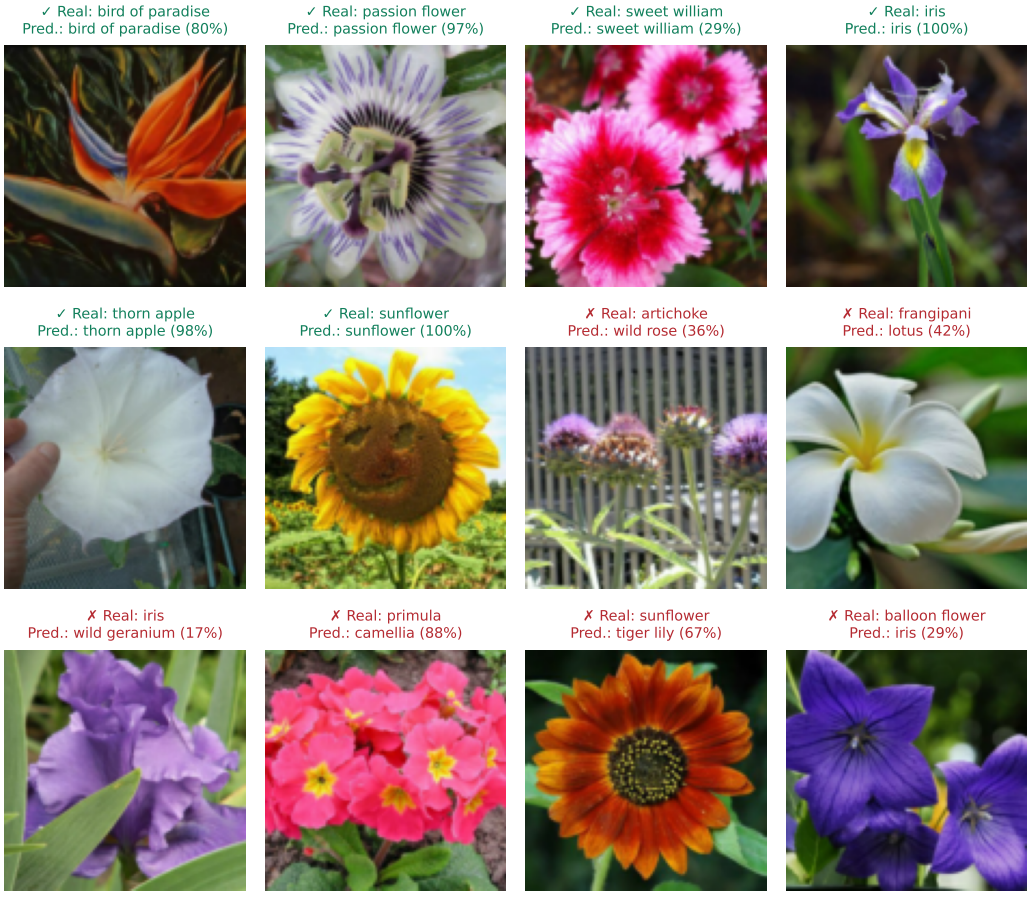


Figura 6: Seis aciertos (verde, ✓) y seis errores (rojo, ×) de la CNN de referencia en validación, elegidos con semilla fija; entre paréntesis, la probabilidad softmax de la clase predicha. La proporción es ilustrativa: la red acierta en 2,378 de 3,712 imágenes.

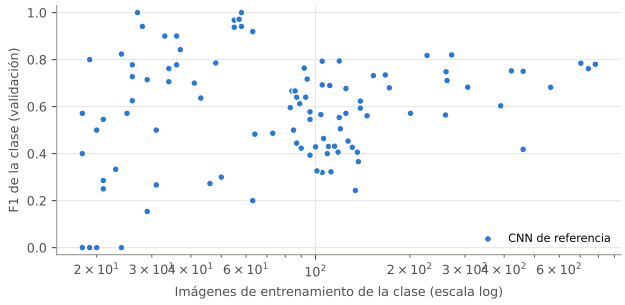


Figura 7: F1 de validación de cada clase frente a su número de imágenes de entrenamiento (escala logarítmica).

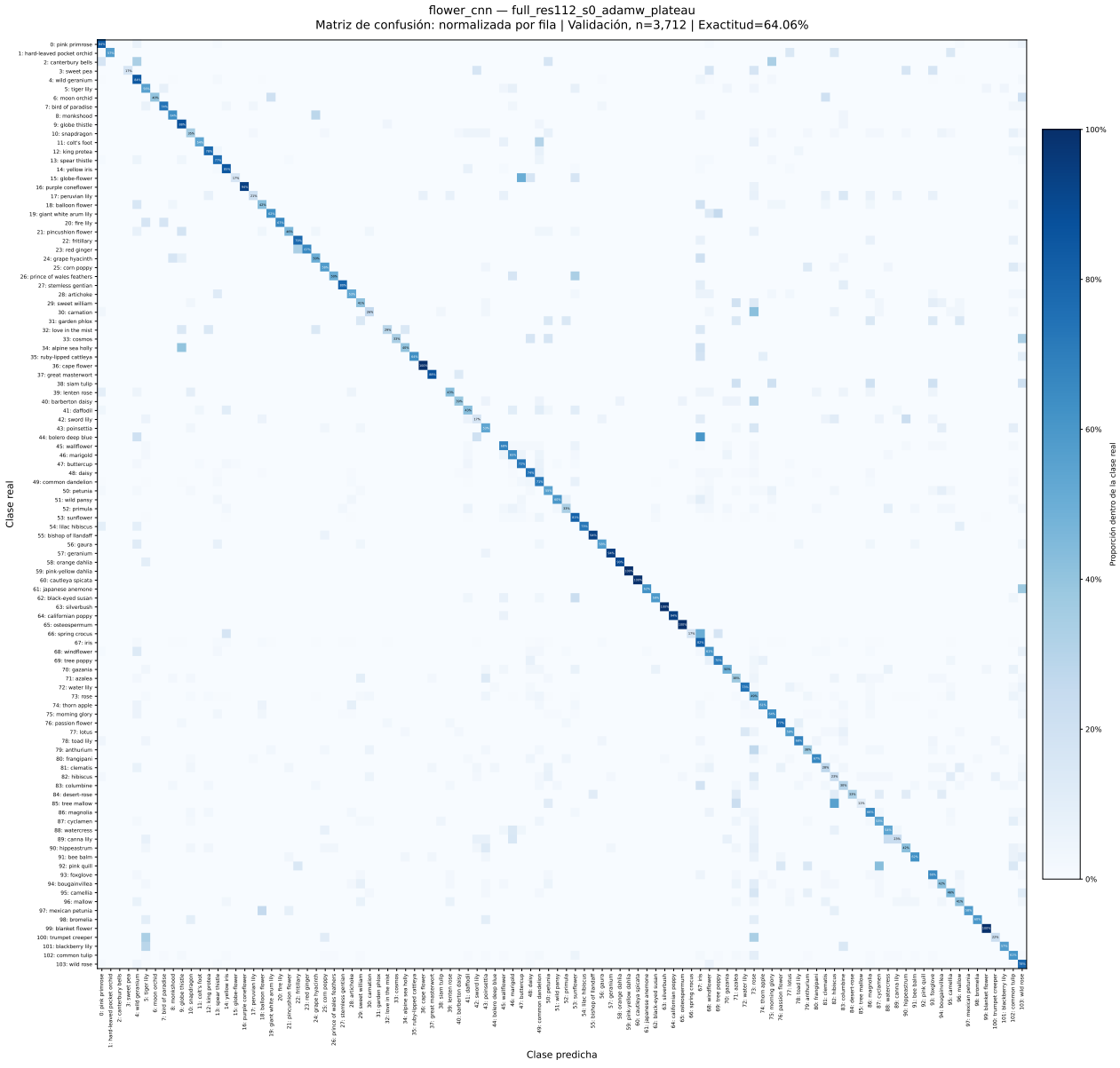


Figura 8: Matriz de confusión normalizada por fila de la CNN de referencia en validación (104 clases).

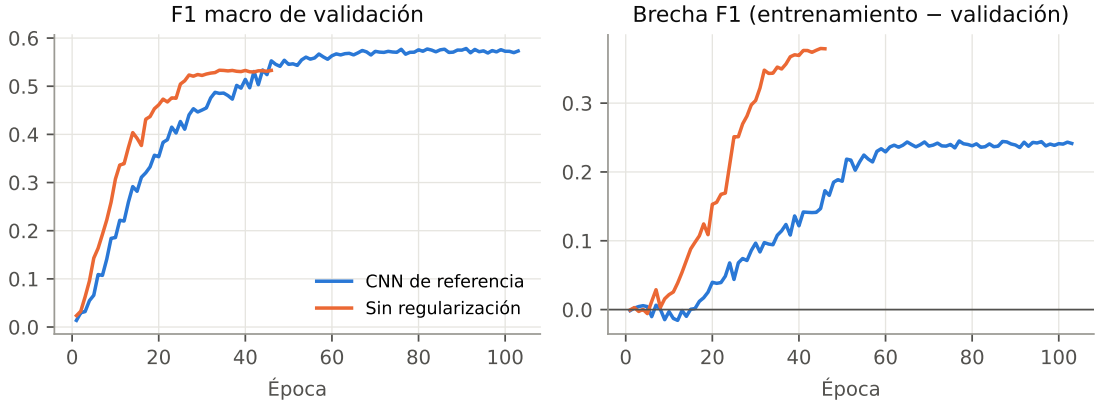


Figura 9: F1 macro de validación y brecha de F1 por época, con y sin regularización (misma semilla, arquitectura y tope de épocas).

Referencias

Bryan Clark. What Is Image Classification? | IBM — ibm.com. https://www.ibm.com/think/topics/image-classification, 2025. [Accessed 28-09-2026].

Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

iNaturalist. iNaturalist: A community for naturalists. https://www.inaturalist.org, 2026. [Accessed 28-09-2026].

Kaggle. Petals to the Metal - Flower Classification on TPU — kaggle.com. https://www.kaggle.com/competitions/tpu-getting-started/overview, s.f. [Accessed 28-09-2026].

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, pages 722–729. IEEE, 2008. doi: 10.1109/ICVGIP.2008.47.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56): 1929–1958, 2014.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016. doi: 10.1109/CVPR.2016.308.